

Building Detection from Satellite Imagery

Assignment 2 — YOLOv8 Object Detection

SpaceNet Paris Dataset (YOLO Format) | Vaibhavi Singh 251159

1. Problem

This project builds an automated building detection system for satellite imagery using deep learning. Detecting buildings from aerial images has real-world applications in urban planning, disaster response, population estimation, and infrastructure monitoring. Manual annotation of satellite images is slow and expensive, making an automated YOLOv8-based pipeline highly valuable for scaling these tasks.

2. Data

The dataset used is the SpaceNet 2 Paris Buildings dataset in YOLO format, sourced from Kaggle (pallav23/icg-spacenet-sandbox-yolo). It contains 1148 RGB satellite image tiles of Paris (.png) along with pre-annotated building bounding boxes in YOLO format (.txt label files). Each label file contains one line per building: class x_center y_center width height (all normalized 0-1). The dataset was split 80/10/10 into train/val/test using sklearn's train_test_split.

Split	Images	Background Images
Train	918	411
Validation	115	54
Test	115	~54
Total	1148	—

The training set has an average of 14.25 buildings per image, ranging from 0 to 82. 411 out of 918 training images contain no buildings (background/vegetation tiles), which presents a class imbalance challenge for the model.

3. Model

YOLOv8 (You Only Look Once, version 8) is a state-of-the-art real-time object detection model developed by Ultralytics. It uses a single-pass convolutional neural network to simultaneously predict bounding boxes and class probabilities, making it significantly faster than two-stage detectors. The architecture includes a CSPDarknet backbone, a Feature Pyramid Network neck for multi-scale detection, and a decoupled detection head.

We selected **YOLOv8s (small)** as a balance between speed and accuracy — large enough to learn complex building shapes from satellite imagery, but small enough to train efficiently on a T4 GPU.

Property	Value
Model variant	YOLOv8s
Total parameters	11,135,987
GFLOPs	28.6
Pretrained on	COCO dataset (80 classes)
Number of classes (finetuned)	1 (building)
Input image size	640 x 640

4. Training

The model was fine-tuned from pretrained COCO weights using the Ultralytics training pipeline. Transfer learning was used because COCO pretrained weights already encode general visual features (edges, shapes, textures) that transfer well to satellite imagery, allowing faster convergence and better results with a relatively small dataset.

Hyperparameter	Value
Epochs	30
Batch size	16
Image size	640
Optimizer	AdamW (auto-selected)
Learning rate	0.002
GPU	Tesla T4 (14 GB)
Training time	~0.216 hours (~13 minutes)
Patience (early stopping)	10 epochs
Final Train box_loss	1.2187
Final Val box_loss	1.3193

5. Results

The model was evaluated on the validation set using the best checkpoint. The training curves showed consistent improvement in mAP@50 from 0.679 at epoch 1 to 0.805 by epoch 26, with both train and validation losses decreasing steadily — indicating healthy learning without significant overfitting.

Metric	Value	What it means
mAP@50	0.8022	Model detects buildings correctly 80.2% of the time at IoU=0.5
Precision	0.8126	Of all predicted boxes, 81.3% were actual buildings
Recall	0.7370	Of all real buildings, the model found 73.7% of them
F1 Score	0.7730	Balanced score between precision and recall
MAE (count)	1.50	On average, predicted count differs from true count by 1.5 buildings

The Precision-Recall curve confirmed a strong area under curve of 0.805. The confusion matrix showed 1357 correct detections, 394 false positives, and 377 false negatives. The end-to-end pipeline processes a single image in approximately 30-50ms on GPU.

6. Observations

What worked well:

Transfer learning from COCO weights gave the model a strong starting point, achieving mAP@50 above 0.67 even in the very first epoch. The model handled images with varying building densities well, with a mean absolute error of only 1.5 buildings per image on the validation set — an improvement over randomly initialized weights. Training converged quickly in under 13 minutes.

What the model struggled with:

The large number of background-only images (411 out of 918 training images) created a class imbalance that occasionally caused false negatives in partially vegetated scenes. The model also struggled with very dark or near-black image tiles (likely cloud cover or no-data regions), and with densely packed buildings where rooftops overlap or touch at the edges.

One idea to improve it:

Filtering out completely black/empty image tiles before training would reduce noise and improve precision. Additionally, using a larger model variant (YOLOv8m) with a higher confidence threshold or test-time augmentation (TTA) would likely push mAP above 0.85. Data augmentation techniques specifically designed for satellite imagery (random rotations, multi-scale crops) could also help.